

Tendências de Motores de Jogos

Semana 9

UMG, Widgets, Binding de Dados e HUD

Disciplina: Tendências de Motores de Jogos (IN46A)

Instituição: IFMS — Campus Dourados

Curso: Tecnologia em Jogos Digitais

Módulo 3 — Resolver Problemas

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

OBJETIVOS

- Compreender UMG como sistema universal de interface em tempo real e o binding de dados como mecanismo de exibição de estado de gameplay
- Criar um Widget simples vinculado a uma variável de gameplay já existente
- Compreender HUD como camada de organização de múltiplos Widgets
- Propor e implementar, com autonomia própria, um HUD que combine os dados de gameplay que o grupo julgar relevantes

Como a semana está organizada



Encontro 1

UMG, Widget Blueprint e binding de dados — exibindo a vida do `HealthComponent`



Encontro 2

HUD — desafio: cada grupo escolhe quais dados compõem seu próprio HUD

Tendências de Motores de Jogos

ENCONTRO 1

UMG, Widget e Binding de Dados

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



A vida do seu personagem já existe no projeto. Mas o jogador consegue vê-la?

Pense na diferença entre o dado de gameplay existir internamente e ser comunicado ao jogador em tempo real.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Separar o dado de gameplay da sua representação visual

- O `HealthComponent` já sabe quanto de vida existe — isso é dado de gameplay, não interface
- Falta uma camada que traduza esse dado em algo visível ao jogador, em tempo real
- Se a interface escrever diretamente na lógica de gameplay, as duas camadas se acoplam
- Toda engine madura resolve isso com uma camada de apresentação de UI separada, atualizada automaticamente

DICA

Trocar o layout ou o estilo visual da interface nunca deveria exigir tocar na lógica do `HealthComponent`.

UMG: a camada de interface em tempo real

- Unreal Motion Graphics — sistema de construção de interface da Unreal
- Widget Blueprint organiza elementos visuais (texto, barra, imagem) sobre um Canvas Panel
- Binding conecta uma propriedade do elemento visual a uma função que lê o dado de gameplay
- O sistema de gameplay nunca precisa disparar a atualização manualmente

Widget Blueprint e Canvas Panel

- Widget Blueprint: asset de interface, análogo a uma "cena" de UI
- Canvas Panel: painel que posiciona elementos livremente na tela
- Elementos básicos: Text, Progress Bar, Image
- Cada elemento pode ter uma ou mais propriedades vinculadas por binding

Interface em tempo real: Unreal × Unity

Unreal

UMG — Widget Blueprint com Canvas Panel; binding declarativo conecta a propriedade visual à função de leitura do dado

Unity

uGUI (Canvas + componentes C#) ou UI Toolkit (UXML/USS); atualização normalmente exige código explícito lendo a variável e escrevendo no componente de UI

Do dado de gameplay ao elemento visual



Demonstração: Widget com binding à vida

O professor cria um Widget Blueprint, monta uma Progress Bar sobre um Canvas Panel, e vincula por binding o valor da barra à vida atual do `HealthComponent`.

Resultado esperado: a barra se atualiza automaticamente em tempo real conforme a vida do personagem muda durante o gameplay.

Imagem sugerida

Objetivo: mostrar o UMG UI Designer com um Widget Blueprint em edição.

Enquadramento: captura de tela do editor da Unreal, painel Designer em primeiro plano.

Elementos presentes: hierarquia do Widget à esquerda (Canvas Panel > Progress Bar), área de preview central, painel Details à direita mostrando o binding configurado.

Destaque visual: circular ou realçar o botão de binding na propriedade "Percent" da Progress Bar.

Legenda sugerida: "Binding conectando a Progress Bar à vida do HealthComponent."

Boas práticas

✓ BOAS PRÁTICAS

- Nomear o Widget Blueprint com o prefixo `WBP_` (conforme PROJECT_ARCHITECTURE.md)
- Usar binding para leitura de dados de gameplay, nunca referência direta escrita a partir do Widget
- Manter cada Widget responsável por um único propósito de exibição

Laboratório do Encontro 1

Cada grupo cria seu próprio Widget Blueprint e vincula por binding um elemento visual simples a uma variável de gameplay já existente no projeto (vida, ou outro dado disponível).

OBJETIVOS

Critério de sucesso: o Widget é exibido em tela durante o gameplay e se atualiza automaticamente quando o dado muda, sem alterar a lógica de nenhum sistema do Módulo 2.

Fechando o Encontro 1

- UMG separa o dado de gameplay da sua representação visual; binding conecta os dois automaticamente
- Cada grupo com um Widget funcional, exibindo em tempo real um dado já existente no projeto
- Próximo encontro: organizar múltiplos Widgets em um único HUD, com liberdade de escolha dos dados

Tendências de Motores de Jogos

ENCONTRO 2

HUD e o Desafio de Comunicação com o Jogador

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tendências de Motores de Jogos



Um Widget por dado resolve um jogo real, com vários dados ao mesmo tempo?

Pense no que acontece na tela quando cada dado de gameplay tem seu próprio Widget solto, sem organização.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

HUD: organização de múltiplos Widgets

- Widget "container", posicionado permanentemente em tela durante o gameplay
- Agrupa e organiza outros elementos ou bindings em um layout único
- Reutiliza o mesmo mecanismo de binding do Encontro 1, aplicado a múltiplos dados
- Não é um conceito tecnicamente novo — é organização do que já foi aprendido

DICA

O HUD não substitui o Widget do Encontro 1 — ele organiza vários Widgets (ou bindings) em um único ponto coerente da tela.

Quais dados já existem para compor o HUD?

- Vida — `HealthComponent` (Semana 8)
- Progresso — `SaveComponent` (Semana 7)
- Estado de jogo — `BP_GameState`
- Outros dados que cada grupo já tenha implementado no próprio projeto

HUD: Unreal × Unity

Unreal

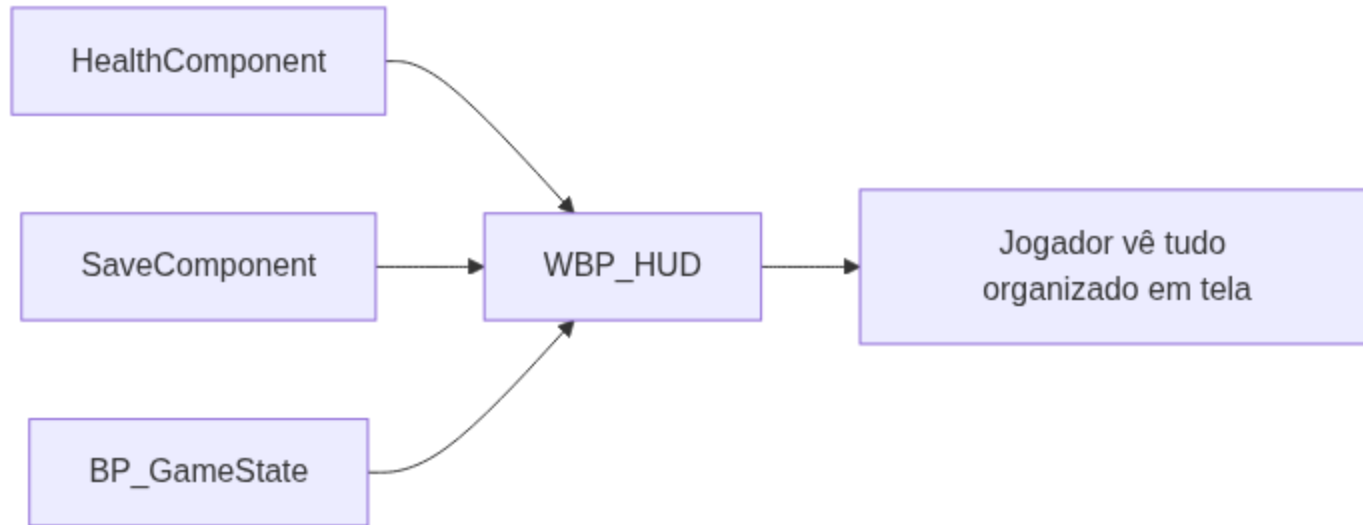
HUD como Widget dedicado, convenção explícita para o Widget de nível superior do gameplay

Unity

Sem classe nomeada equivalente; mesmo resultado obtido organizando um Canvas raiz (uGUI) ou documento UXML raiz (UI Toolkit), geralmente mantido por um script de gerenciamento de UI

Tendências de Motores de Jogos

Vários dados, um único HUD



IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Demonstração: HUD combinando dois dados

O professor monta um `WBP_HUD` que combina, em um único Canvas Panel, dois elementos já conhecidos do Encontro 1 (por exemplo, vida e um segundo dado simples), mostrando que o HUD é apenas organização, não um conceito novo.

Resultado esperado: a turma vê os dois dados exibidos juntos, de forma organizada, antes de decidir sozinha quais dados e layout usar no próprio desafio.

Imagem sugerida

Objetivo: mostrar um exemplo de HUD combinando dois elementos de interface durante o gameplay.

Enquadramento: captura de tela em modo Play, viewport completa.

Elementos presentes: barra de vida no canto superior esquerdo, um segundo indicador (ex.: contador de progresso) próximo, sem sobreposição.

Destaque visual: contorno leve ao redor da área do HUD para diferenciar da cena de jogo.

Legenda sugerida: "WBP_HUD combinando vida e progresso em um único layout organizado."

Desafio: HUD próprio de cada grupo

Cada grupo decide quais dados de gameplay já existentes (vida, itens, progresso) devem compor o HUD, propõe a própria solução visual e de binding, e apresenta ao final a justificativa técnica das escolhas.

ATENÇÃO

Não há indicação prévia de quais dados exibir nem do layout. A escolha faz parte do desafio.

Boas práticas

✓ BOAS PRÁTICAS

- Nomear o HUD com o prefixo `WBP_` (ex.: `WBP_HUD`), conforme `PROJECT_ARCHITECTURE.md`
- Exibir apenas os dados que o jogador precisa ver constantemente — evitar poluição visual
- Justificar a escolha dos dados e do layout a partir da necessidade de comunicação com o jogador, não da preferência pessoal

Resumo da Semana 9

- UMG separa o dado de gameplay da sua representação visual; binding conecta os dois automaticamente
- Widget expõe um dado isolado; HUD organiza múltiplos Widgets em uma camada coerente
- Segundo desafio de liberdade real de solução da disciplina, com escolha justificada de dados e layout
- Nenhum sistema do Módulo 2 substituído ou quebrado

Checklist da Semana

OBJETIVOS

- Widget funcional com ao menos um elemento vinculado por binding (Encontro 1)
- WBP_HUD funcional, combinando ao menos dois dados de gameplay (Encontro 2)
- Escolha dos dados e do layout justificada tecnicamente pelo grupo
- Nenhum sistema do Módulo 2 substituído ou quebrado

Próximos passos



A Semana 10 introduz o Inventário, reutilizando os itens modelados em Data Table/Struct na Semana 6, e amplia o Interaction System da Semana 5 para suportar múltiplos tipos de interação (coletar, usar, descartar). O `WBP_HUD` construído nesta semana será o destino natural da exibição dos itens do inventário.

Leitura recomendada: UMG UI Designer (Epic Games Documentation).